

1 - Multi-Agent Interactions

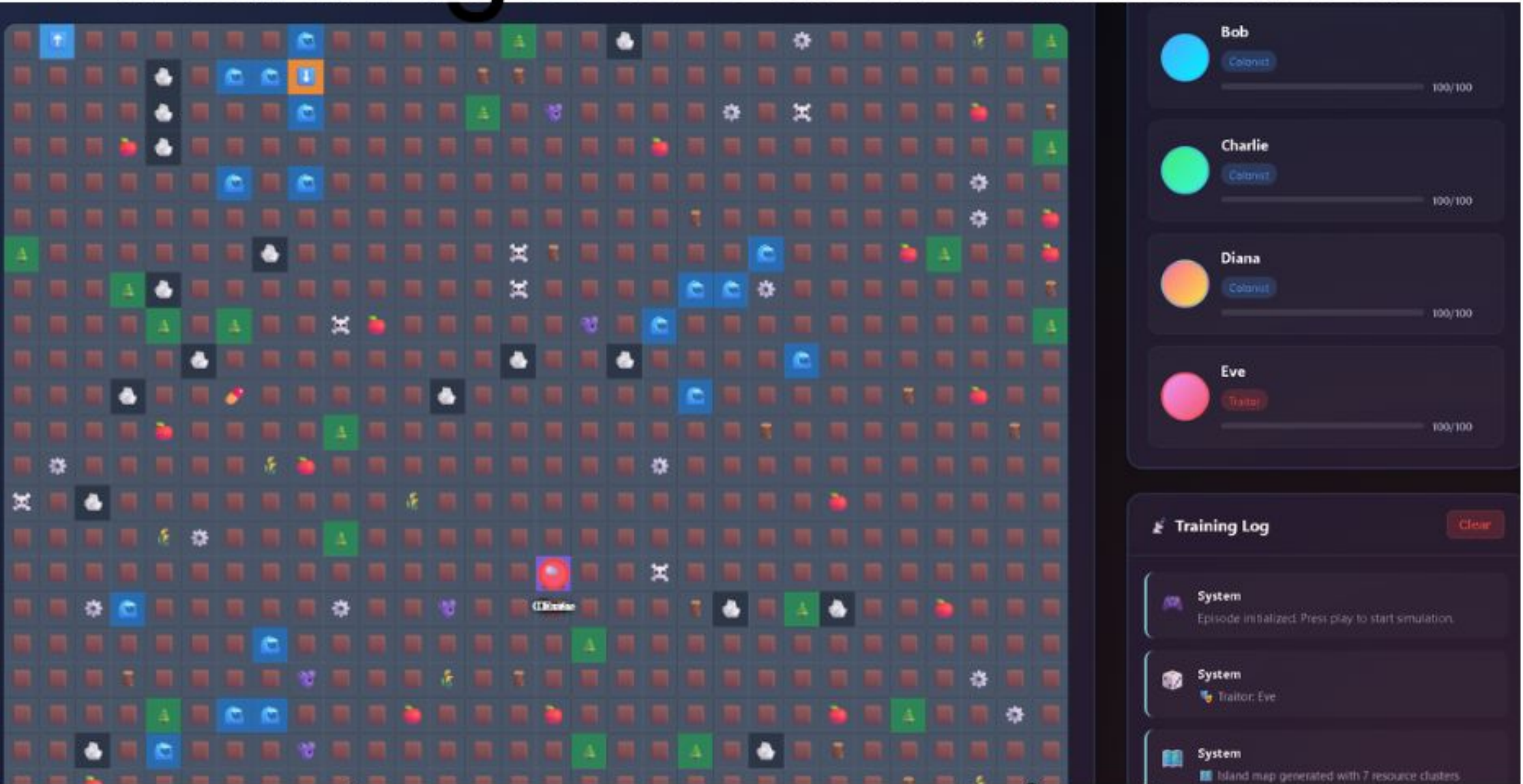
Multiple agents competing / collaborating in an environment.

Environments for this theme involve cooperation, competition, negotiation, and coalition formation. Learning from these environments will enable agents to model the beliefs and incentives of others in partially observable settings. This drives theory-of-mind reasoning and emergent strategic behavior.

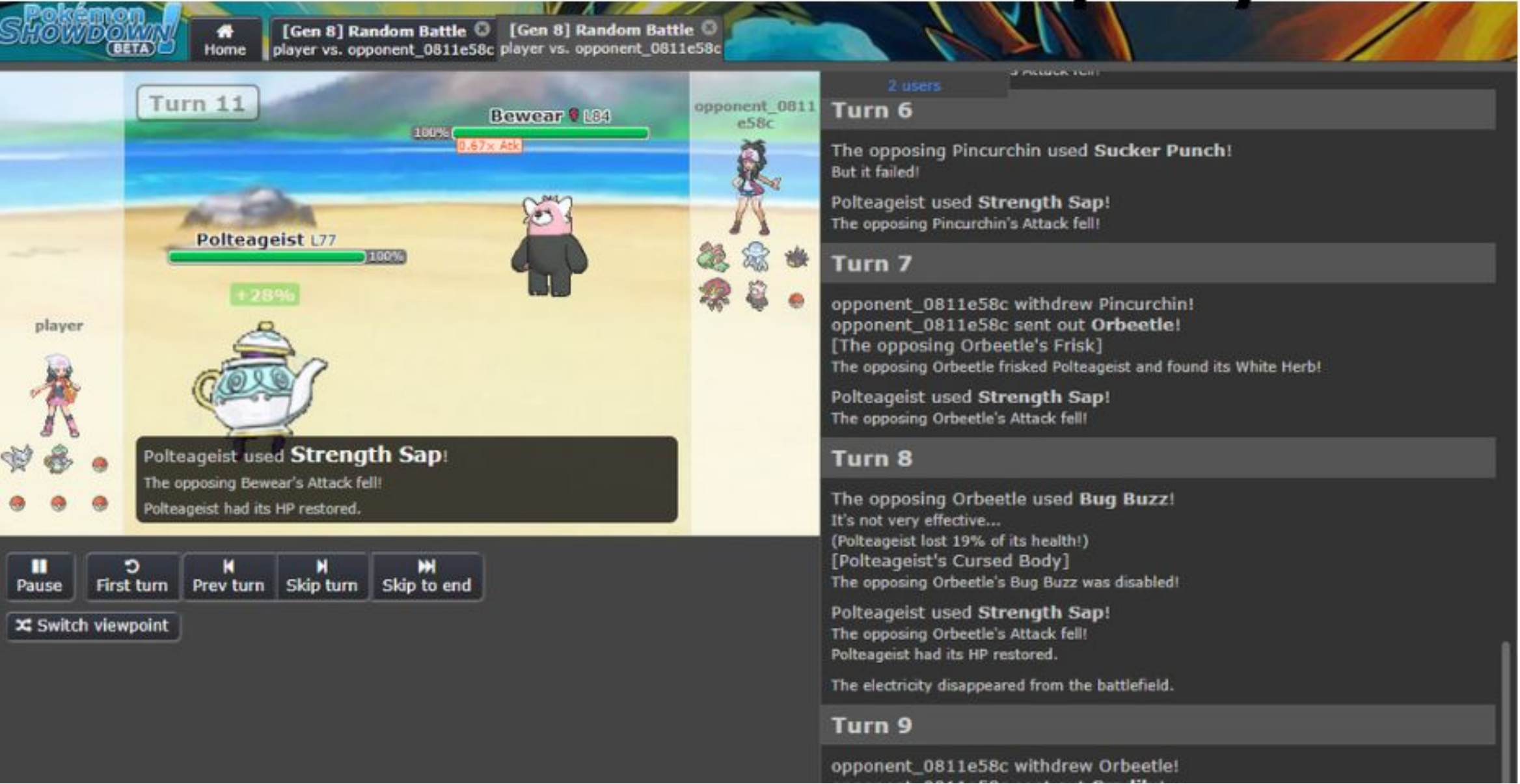
Expected Outcome: an environment that can be used to train multi-agent task handling in a LLM

Example environments: Market simulations, compute-allocation negotiations, collaborative puzzle worlds, mixed cooperative/competitive strategy games.

Multi Agent Survival islands



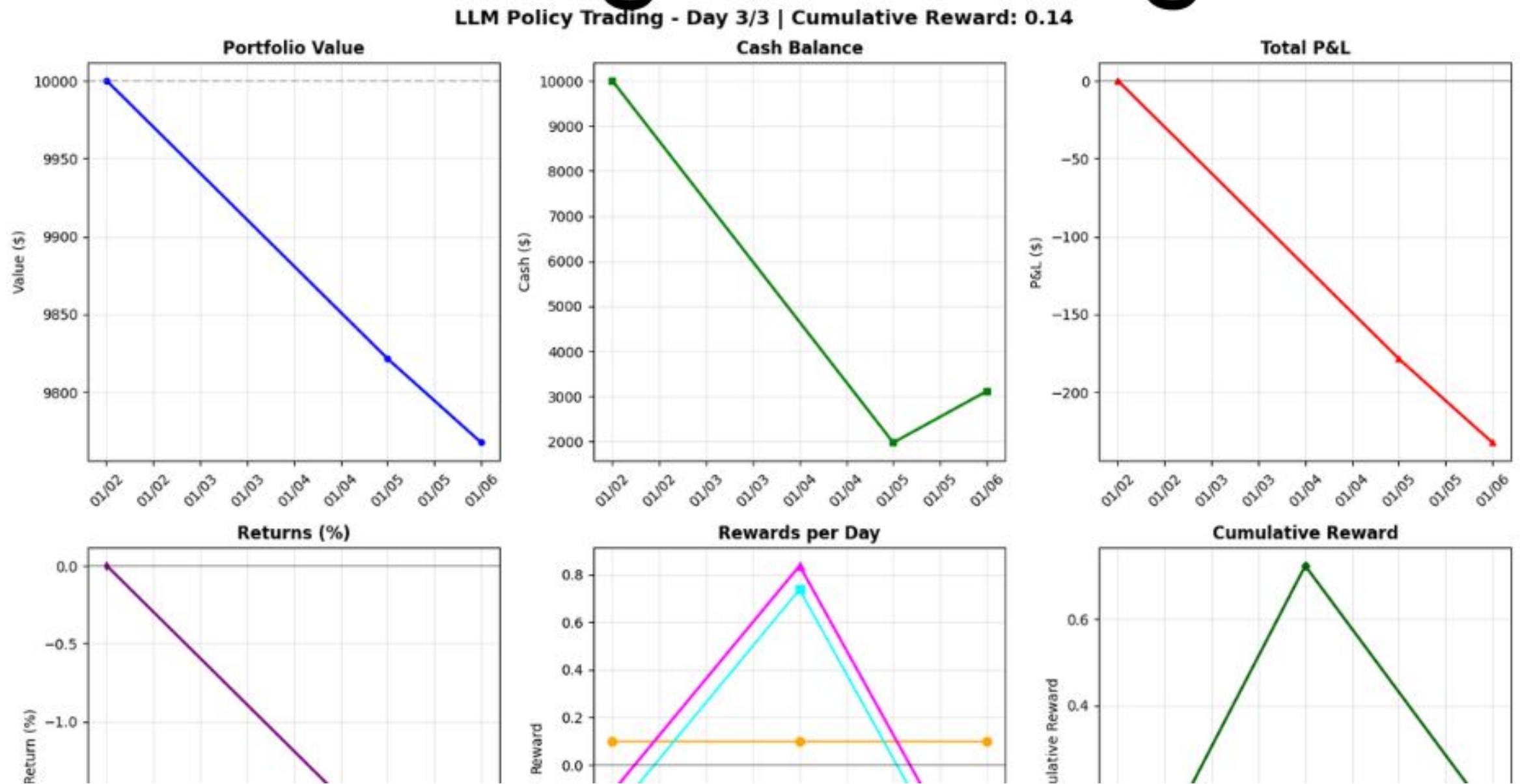
Pokemon - multi player



Strategy games



Multi agent trading



2 - Long-Horizon Planning & Instruction Following

Super long reasoning RL - 300 instructions, tool calls etc

You will build environments that require deep, multi-step reasoning with sparse or delayed rewards. After using these environments, the goal is to enable agents to decompose goals, track state over extended trajectories, and recover from early mistakes. The aim is to push beyond shallow next-token reasoning toward structured planning and durable internal representations.

Expected Outcome: an environment that can capture and improve LLM behaviour on challenging long horizon tasks that need long running sessions beyond context memory limits.

Example environments: Research-planning simulators, large-scale codebase refactoring tasks, strategic resource management worlds, long-horizon logistics optimization, extremely complicated long-horizon instruction following (e.g., 300 instructions scattered around).

Needles: 8 (Hard)



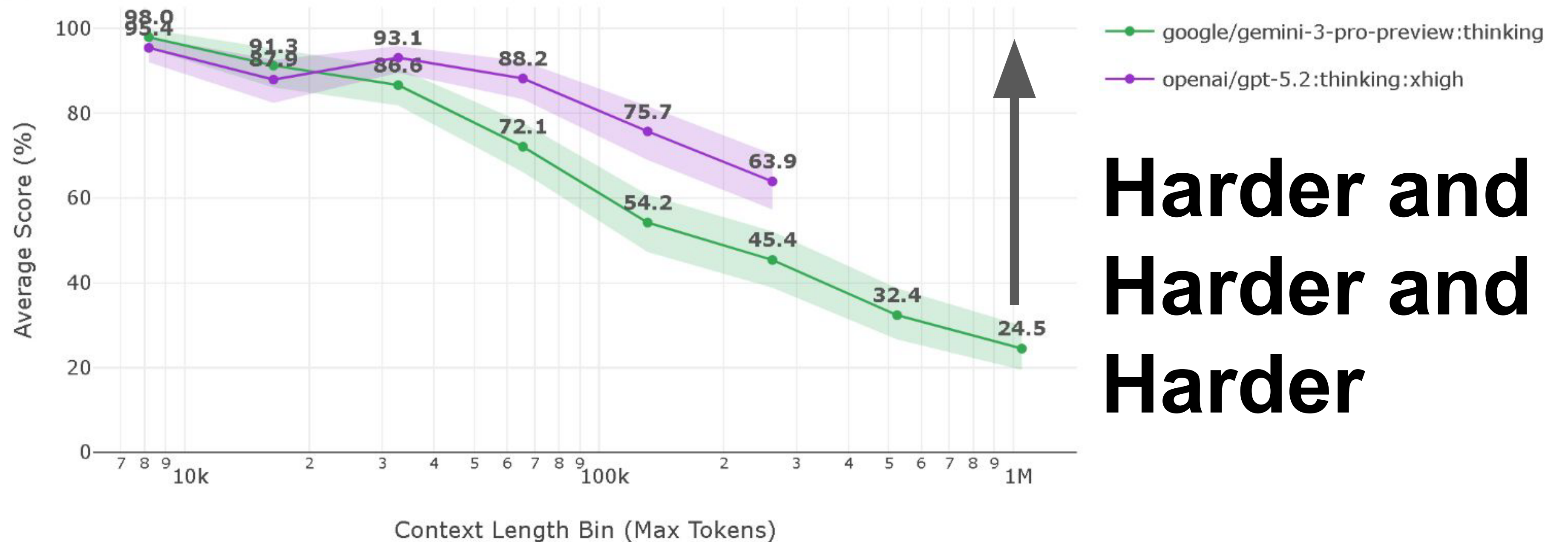
Chart Options

Line

Bar

☐ Use Random Colors☒ Show Chart Labels☒ Show CI 95%

OpenAI-MRCR: Top Performers (8-Needle)



Legend Interaction: Single-click to toggle individual models. Double-click to toggle all models from the same provider.

Harder and
Harder and
Harder

3.1 - World Modeling - Professional Tasks

Desktop application environments, enterprise applications, computer use agents

Here you will develop environments that require real interaction with tools, APIs, or dynamic systems where the model is expected to do real hard work instead of exploiting short-cuts to arrive at the desired outcome. Learning from these environments will enable agents to maintain consistent internal state, update beliefs based on outcomes, and orchestrate multi-step workflows. The goal is to strengthen causal reasoning and persistent world models.

Expected Outcome: an environment capturing nuances of a defined partially observable world and improve LLM interaction with it

Example environments: Dynamic browser/API ecosystems, enterprise applications, scientific workflow loops (papers → code → experiments), economic simulations with feedback, tool-discovery benchmarks.

Program Design & Evaluation Toolkit

Target population	Recently incarcerated adults (25-45 years)
-------------------	--

Annual capacity	100
Program duration	13 weeks intensive + 6 months support

Program duration	12 weeks intensive + 6 months support
Geographic focus	Urban areas with high unemployment

Geographic focus	Urban areas with high unemployment
Key outcomes	Job placement, economic stability, rec

1997	1998	1999	2000	2001	2002	2003	2004	2005	2006	2007	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021	2022	2023	2024	2025	2026	2027	2028	2029	2030	2031	2032	2033	2034	2035	2036	2037	2038	2039	2040	2041	2042	2043	2044	2045	2046	2047	2048	2049	2050	2051	2052	2053	2054	2055	2056	2057	2058	2059	2060	2061	2062	2063	2064	2065	2066	2067	2068	2069	2070	2071	2072	2073	2074	2075	2076	2077	2078	2079	2080	2081	2082	2083	2084	2085	2086	2087	2088	2089	2090	2091	2092	2093	2094	2095	2096	2097	2098	2099	2100	2101	2102	2103	2104	2105	2106	2107	2108	2109	2110	2111	2112	2113	2114	2115	2116	2117	2118	2119	2120	2121	2122	2123	2124	2125	2126	2127	2128	2129	2130	2131	2132	2133	2134	2135	2136	2137	2138	2139	2140	2141	2142	2143	2144	2145	2146	2147	2148	2149	2150	2151	2152	2153	2154	2155	2156	2157	2158	2159	2160	2161	2162	2163	2164	2165	2166	2167	2168	2169	2170	2171	2172	2173	2174	2175	2176	2177	2178	2179	2180	2181	2182	2183	2184	2185	2186	2187	2188	2189	2190	2191	2192	2193	2194	2195	2196	2197	2198	2199	2200	2201	2202	2203	2204	2205	2206	2207	2208	2209	2210	2211	2212	2213	2214	2215	2216	2217	2218	2219	2220	2221	2222	2223	2224	2225	2226	2227	2228	2229	2230	2231	2232	2233	2234	2235	2236	2237	2238	2239	2240	2241	2242	2243	2244	2245	2246	2247	2248	2249	2250	2251	2252	2253	2254	2255	2256	2257	2258	2259	2260	2261	2262	2263	2264	2265	2266	2267	2268	2269	2270	2271	2272	2273	2274	2275	2276	2277	2278	2279	2280	2281	2282	2283	2284	2285	2286	2287	2288	2289	2290	2291	2292	2293	2294	2295	2296	2297	2298	2299	2300	2301	2302	2303	2304	2305	2306	2307	2308	2309	2310	2311	2312	2313	2314	2315	2316	2317	2318	2319	2320	2321	2322	2323	2324	2325	2326	2327	2328	2329	2330	2331	2332	2333	2334	2335	2336	2337	2338	2339	2340	2341	2342	2343	2344	2345	2346	2347	2348	2349	2350	2351	2352	2353	2354	2355	2356	2357	2358	2359	2360	2361	2362	2363	2364	2365	2366	2367	2368	2369	2370	2371	2372	2373	2374	2375	2376	2377	2378	2379	2380	2381	2382	2383	2384	2385	2386	2387	2388	2389	2390	2391	2392	2393	2394	2395	2396	2397	2398	2399	2400	2401	2402	2403	2404	2405</
------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	------	--------

Component	Description
(1) Non-Resident Landlord (NRL)	(2) Rental Income from Residential Property (RIRP)

Outcome measure	Target	Measurement point
Program completion rate	80%	Week 12

Executive Dashboard



Delete Sheet

Delete Sheet

3.2 - World Modeling - Personalized Tasks

Calendar, planning agents, email, messages etc

Here we will develop an environment that offers real personalised task handling, imagine replying to personal messages or handling dinner conflicts due to work conflicts, replying to tough emails. Think any personal assistant tasks

Expected Outcome: An environment that gives the model a realistic simulation of handling personal tasks, conflicts and managing them as delegations

Example environments: Executive Assistant Meeting Planner, Dinner and drive planning, email and message replying, etc

[https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/FunctionGemma_\(270M\)-Mobile-Actions.ipynb](https://colab.research.google.com/github/unslothai/notebooks/blob/main/nb/FunctionGemma_(270M)-Mobile-Actions.ipynb)

FunctionGemma_(270M)-Mobile-Actions.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Copy to Drive

Connect T4

```
[ ] from unsloth.chat_templates import train_on_responses_only
trainer = train_on_responses_only(
    trainer,
    instruction_part = "<start_of_turn>user\n",
    response_part = "<start_of_turn>model\n",
)
```

Map (num_proc=12): 100% 9604/9604 [00:03<00:00, 5849.18 examples/s]

Map (num_proc=12): 100% 50/50 [00:00<00:00, 13.30 examples/s]

We print the full un-masked string:

```
[ ] tokenizer.decode(trainer.train_dataset[100]["input_ids"])
```

<bos><bos><start_of_turn>developer\nCurrent date and time given in YYYY-MM-DDTHH:MM:SS format: 2026-07-28T11:21:51\nDay of week is Tuesday\nYou are a model that can do function calling with the following functions<start_function_declaration>declaration:turn_on_flashlight{description:<escape>Turns the flashlight on.<escape>,parameters:{properties:{body:{description:<escape><escape>,type:<escape><escape>},datetime:{description:<escape><escape>,type:<escape><escape>},email:{description:<escape><escape>,type:<escape><escape>},first_name:{description:<escape><escape>,type:<escape><escape>},last_name:{description:<escape><escape>,type:<escape><escape>},phone_number:{description:<escape><escape>,type:<escape><escape>},query:{description:<escape><escape>,type:<escape><escape>},subject:{description:<escape><escape>,type:<escape><escape>},title:{description:<escape><escape>,type:<escape><escape>},to:{description:<escape><escape>,type:<escape><escape>}},type:<escape>OBJECT<escape>}}<end_function_

See if we did the masking correctly - "-" is the mask

```
[ ] [tokenizer.decode([tokenizer.pad_token_id if x == -100 else x for x in trainer.train_dataset[100]["labels"]]).replace(tokenizer.pad_token, "-")]
```

['------<start_function_call>call:create_calendar_event{body:None,datetime:2026-08-03 09:30:00,email:None,first_name:None,last_name:None,phone_number:None,query:None,subject:None,title:<escape>Team Sync Meeting<escape>,to:None}<end_function_call><start_function_response>']

Show current memory stats

Show code

GPU = Tesla T4. Max memory = 14.741 GB.
0.832 GB of memory reserved.

Let's train the model! To resume a training run, set `trainer.train(resume_from_checkpoint = True)`

trainer_stats = trainer.train()

... ==((====))== Unsloth - 2x faster free finetuning | Num GPUs used = 1
\\ // Num examples = 9,604 | Num Epochs = 1 | Total steps = 100
O^O/ _/ \ Batch size per device = 4 | Gradient accumulation steps = 2
\\ ____/ Data Parallel GPUs = 1 | Total batch size (4 x 2 x 1) = 8
"-_____" Trainable parameters = 3,796,992 of 271,895,168 (1.40% trained)
Unsloth: Will smartly offload gradients to save VRAM!
[100/100 15:50, Epoch 0/1]

Step	Training Loss	Validation Loss
------	---------------	-----------------

Variables

Terminal

4 - Self Improvement

Automating RL - auto tasks

The focus here is to create environments where agents can learn to generate new challenges, escalate difficulty, and improve through self-play or adaptive curricula. Rather than optimizing fixed tasks, the goal is for agents to learn to drive their own capability growth. The objective is recursive skill amplification.

Expected Outcome: an environment for improving self-play of a LLM over a defined set of tasks

Example environments: Self-play negotiation arenas, auto-generated math/proof tasks, evolving coding competitions, adaptive RL curricula.

5 - Wild Card - Impress us!